Replicating the use of historical data and bookmakers odds in modeling football scores

Gilbert Annersten [*1] **Fabian Lindh** [*1] **William Jarkander** [*1]

Abstract

This study replicates a Bayesian hierarchical model derived from "*Combining historical data and bookmakers odds in modeling football scores*" by Egidi et al. (2018). The paper proposes a new model that combines historical match data with bookmakers odds to estimate and predict match results. The project report replicates the study for the Premier League, one of the four leagues analyzed in the paper. The implementation gave results very similar to those of the article.

1. Introduction

The study discusses how earlier studies have commonly used historical match data and/or compared the results with odds for win/lose/draw afterward (Egidi et al., 2018). The statistical problem in this study is instead to incorporate the bookmakers odds for win/draw/lose into models of match result (number of goals per team) to predict and estimate match results. The authors attempt to solve this statistical problem by combining the two sources of information (goals and odds) through a hierarchical Bayesian model. A Bayesian approach to betting is intuitive, modeling goals scored during a game as a "goal rate" parameter in a Poisson process. Since the game has inherent randomness built in, it is impossible to model every aspect of a football game. The model was replicated in Stan, following the article where possible, along with the authors' published code for settings where the article is unspecific, to assess whether the published results could be reproduced.

2. Data

The data consists of both the match scores and three-way bookmaker odds from Europe's top four leagues. The data consists of the seasons 2007-08 through 2016-17 and includes historical odds from seven distinct betting companies (Bet365, Bet&Win, Interwetten, Ladbrokes, Sportingbet, VC Bet and William Hill). The focus is on replicating the results of the Premier League, one of the four leagues analyzed in the study. This information is openly accessible and can be found at the website football-data.co.uk. Since the data is publicly available, we have access to exactly the same data the paper used.

In addition to the match results, the model uses the scoring rates implied by the bookmakers odds as data. These are not directly available and must be derived from the odds. To obtain the probabilities from the bookmakers odds, we need a transformation, since the sum of the inverse of the odds of a single match has to be greater than one in order to guarantee the bookmakers profits. Basic normalization is used: $\pi_i = \frac{1/o_i}{1/o_{\text{win}}+1/o_{\text{draw}}+1/o_{\text{lose}}}$, where o_i is the bookmakers odds for outcome $i \in \{\text{win, draw, lose}\}$. This guarantees that $\sum \pi_i = 1$ for a single match, which removes the bookmakers margin.

Assuming the home and away goals follow independent Poisson distributions, the goal difference is Skellam distributed. For each pair of match m and bookmaker s, to find the implicit scoring rates $\hat{\theta}^s_{m1}, \hat{\theta}^s_{m2}$, we solve

$$\begin{aligned} \pi^s_{win,m} &= P(y_{m1} > y_{m2} | \theta^s_{m1}, \theta^s_{m2}) \\ \pi^s_{draw,m} &= P(y_{m1} = y_{m2} | \theta^s_{m1}, \theta^s_{m2}) \\ \pi^s_{lose,m} &= P(y_{m1} < y_{m2} | \theta^s_{m1}, \theta^s_{m2}) \end{aligned} \tag{1}$$

The authors do not state how this system is solved. According to the authors' code, the rates are obtained numerically by minimizing the squared difference between the bookmaker's basic-normalized probabilities and the corresponding Skellam probabilities for home win, draw, and away win, using R's `optim()` with its default Nelder–Mead algorithm. We follow the same procedure.

$$(\hat{\theta}^s_{m1}, \hat{\theta}^s_{m2}) = \arg \min_{\theta_{m1}, \theta_{m2} > 0} \sum \left(\pi^s_{i,m} - P_i(\theta_{m1}, \theta_{m2})\right)^2 \tag{2}$$

where $i \in \{\text{win, draw, lose}\}$

For the Stan implementation, teams were mapped to integer indices 1–35 and seasons to indices 1–10. The first nine seasons (2007/08–2015/16) form the training data ($\texttt{M} = 3420$

[*]Equal contribution [1]Department of Statistics, Uppsala University, Uppsala, Sweden. Correspondence to: Gilbert Annersten <gilbertannersten01@gmail.com>.

matches), and the 2016/17 season is held out as the test season ($\texttt{M_test} = 380$). For each match, the implied scoring rates of the $S = 7$ bookmakers obtained from Equation 2 are stored as $\texttt{M} \times 7$ matrices, and α is computed as their average across bookmakers.

3. Model

Let y_{m1} and y_{m2} denote the observed goals scored by the home and away team in the mth game; these are modeled as

$$y_{m_j}|\theta_{m_j}, \lambda_{m_j} \sim \text{Poisson}(p_m\theta_{m_j} + (1-p_m)\lambda_{m_j}), j = 1, 2 \tag{3}$$

Each scoring rate is a convex combination of two parameters: θ_{mj}, the scoring intensity estimated from historical match results, and λ_{mj}, the scoring intensity implied by the bookmakers' three-way odds (odds of a win, a draw and a loss). The weight is governed by a match-specific mixing parameter $p_m \sim \text{Beta}(a, b)$ (The authors specify neither a nor b in the article. In their code they used $a = b = 1$; we chose to follow the same procedure).

The historical goal intensities are modeled with a log-linear specification, for both the home and away team for each team $t = 1, 2 \dots T$ as

$$\begin{aligned} \log(\theta_{m_1}) &= \mu + \text{home} + \text{att}_{t[m]1} + \text{def}_{t[m]2} \\ \log(\theta_{m2}) &= \mu + \text{att}_{t[m]2} + \text{def}_{t[m]1} \end{aligned} \tag{4}$$

The index $t[m]$ denotes the team t in the mth game; μ represents the global intercept, and the parameter "home" represents the advantage of a home game. The parameters "att" and "def" represent the attacking and defending strengths. To allow the team strengths to change over time, the attack and defense effects are given a random-walk-with-drift prior across seasons. For each team $t = 1, 2, \dots T$ and season $\tau = 2, 3 \dots, \mathcal{T}$

$$\begin{aligned} \text{att}_{t,\tau} &\sim N(\mu_{att} + \text{att}_{t,\tau-1}, \sigma^2_{att}), \\ \text{def}_{t,\tau} &\sim N(\mu_{def} + \text{def}_{t,\tau-1}, \sigma^2_{def}) \end{aligned} \tag{5}$$

A team's effect in season τ is centered at its previous-season effect plus the drift, with σ^2_{att} and σ^2_{def} controlling how much strengths can change from one season to the next. For the first season ($\tau = 1$) we assume

$$\begin{aligned} \text{att}_{t,1} &\sim N(\mu_{att}, \sigma^2_{att}) \\ \text{def}_{t,1} &\sim N(\mu_{def}, \sigma^2_{def}) \end{aligned} \tag{6}$$

To ensure that the parameters are identified, the attack and defense effects are constrained to sum to zero within each season.

$$\sum_{t=1}^{T} \text{att}_{t,\tau} = 0, \qquad \sum_{t=1}^{T} \text{def}_{t,\tau} = 0, \qquad \tau = 1, \dots, \mathcal{T}. \tag{7}$$

Since the drift terms μ_{att} and μ_{def} are common to all teams, they cancel under the zero-sum constraint and do not enter the model. We keep them to follow the original article.

The parameters μ, home, μ_{att}, μ_{def} are assigned priors $N(0, 10)$, while $\sigma^2_{att}, \sigma^2_{def} \sim$ Inv-Gamma$(0.001, 0.001)$. The team effects are thus estimated from past results. In this part, our model differs from the authors code implemented in JAGS.

Since JAGS parametrizes distributions by their precision, the authors specify μ, home, μ_{att} and μ_{def} as normal with precision 0.0001, i.e. variance $\frac{1}{0.0001} = 10000$, whereas the article states a variance of 10. We chose to follow the article. Furthermore, the authors chose to use $1/\sigma^2_{att}, 1/\sigma^2_{def} \sim$ Gamma$(0.01, 0.01)$, which is equivalent to $\sigma^2_{att}, \sigma^2_{def} \sim$ Inv-Gamma$(0.01, 0.01)$. This differs from the article (0.01 instead of 0.001). Again, we chose to follow the article.

After modeling the matches, we move on to the bookmaker odds. For each match m and bookmaker $s = 1, \dots, S$, the implicit scoring rates $\hat{\theta}^s_{m1}, \hat{\theta}^s_{m2}$ from Equation 2 are treated as observed data. The implied rates are assumed independent across bookmakers and log-normally distributed,

$$\hat{\theta}^1_{mj}, \dots, \hat{\theta}^S_{mj} \sim \text{Log-normal}(\lambda_{mj}, \tau^2_j), \quad j = 1, 2 \tag{8}$$

where λ_{mj} acts as a location parameter. The authors never explain in the article why λ_{mj} is modeled on the log scale; λ_{mj} enters Equation 3 alongside θ_{mj}, which is on the natural scale. The article never addresses this inconsistency. This deflates the bookmakers contribution to the scoring intensities. τ_j is also never given a distribution in the article, but in their JAGS code $\tau_j \sim U(0, 10)$. We chose to implement the same distribution. In turn, λ_{mj} is given a log-normal prior with hyperparameter α_j,

$$\lambda_{mj} \sim \text{Log-normal}(\alpha_j, 10), \quad j = 1, 2 \tag{9}$$

The authors never mention the value of α_j in the article; however, in their JAGS code α_j is the average implied rate across bookmakers, $\frac{1}{S}\sum_s \hat{\theta}^s_{mj}$, which we adopt. This is somewhat unconventional, as the bookmaker data informs both the likelihood and the prior, through α_j stemming from the implied rates $\hat{\theta}^s_{mj}$, which constitutes a double use of the data. In the authors JAGS code the precision of λ_{mj} is 0.01, i.e variance is $\frac{1}{0.01} = 100$. Here, our implemented model differs from the authors model again, since we set the variance in accordance with the paper.

4. Computation

The Stan model was fitted with the same settings as the paper's implementation, using three chains with 5000 iterations. Since the model is replicated in Stan, the

No-U-Turn Sampler (NUTS) is used, while the original paper used Gibbs sampling in JAGS. For each chain, 1000 iterations were discarded as warm-up, the same number as in the article. Unique to our reproduction is that we also set adapt_delta to 0.9 instead of the default 0.8 to reduce the risk of divergences in HMC. The maximum tree depth was also changed from the default 10 to 12, which allows longer trajectories per iteration. With these settings, there were no divergences and no iterations hit the max tree depth.

	mean	sd	ESS	$\hat{R}$
μ	0.686	0.026	7102	1.00
home	0.245	0.023	13533	1.00
μ_{att}	0.0	3.1	20673	1.00
μ_{def}	0.0	3.2	19068	1.00
σ^2_{att}	0.0105	0.0027	4338	1.00
σ^2_{def}	0.0088	0.0026	3859	1.00
τ_1	0.15334	0.00073	15693	1.00
τ_2	0.2994	0.0014	18549	1.00

Table 1. Basic diagnostics of the Stan model for the eight global parameters with digits for mean and sd reported where their respective MCSE is zero

Table 1 shows the diagnostics for the eight global parameters in the model. Using the model fit, we observed no divergences, and none of the 12 000 post-warm-up iterations reached the maximum tree depth of 12. Of all the parameters, both global and total, all achieved a $\hat{R}$ of lower than 1.01, which indicates convergence of the chains. All global parameters also achieved a fairly high ESS. Since μ_{att} and μ_{def} do not affect the model, in accordance with our earlier discussion, the resulting posteriors should match their priors. The original article does not report numerical estimates for the model parameters. A direct parameter comparison table is therefore not possible.

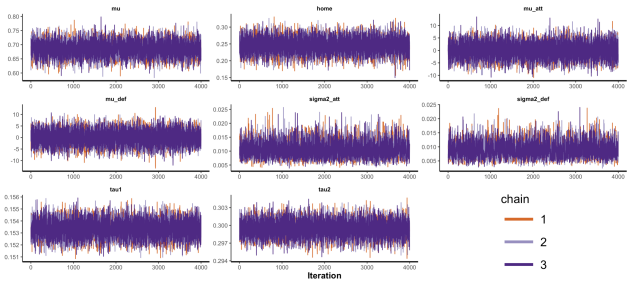


Figure 1. Trace plots for the eight global parameters after warm-up

The trace plots for the global parameters in Figure 1 indicate convergence. All chains after warm-up seem to overlap for each parameter; and there appear to be no visible trends.

5. Reproduction results

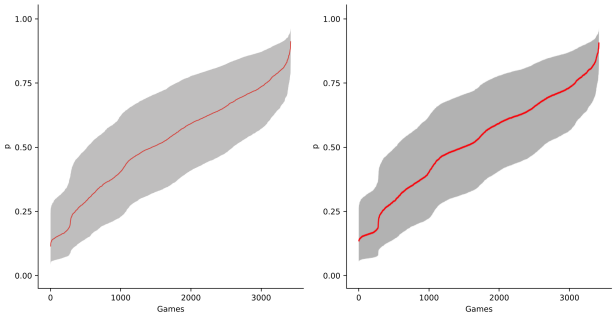
Team	P(1st)	P(2nd)	P(3rd)	Actual Rank
Chelsea	0.1096	0.1360	0.1232	1
Tottenham	0.1104	0.1276	0.1424	2
Man City	0.3564	0.2048	0.1444	3

Table 2. Posterior probabilities top 3 EPL 16/17, Original article

Team	P(1st)	P(2nd)	P(3rd)	Actual Rank
Chelsea	0.11 (0.003)	0.13 (0.003)	0.14 (0.003)	1
Tottenham	0.10 (0.003)	0.13 (0.003)	0.14 (0.003)	2
Man City	0.37 (0.004)	0.22 (0.004)	0.14 (0.003)	3

Table 3. Posterior probabilities top 3 EPL 16/17, Our replication

Tables 2 and 3 present the predicted probabilities of finishing first, second and third for the EPL 2016/17, with MCSEs in parentheses for our replication. The authors did not report MCSEs, so the precision of their estimates is unknown. Rounded to two decimals, four of the nine probabilities coincide, and the remaining five differ by 0.01–0.02. Given our MCSEs, these differences cannot be explained by our Monte Carlo error alone. They may stem from the unreported MCSE of the original, the prior differences between the authors' JAGS code and the article, which we follow, or a combination of both.

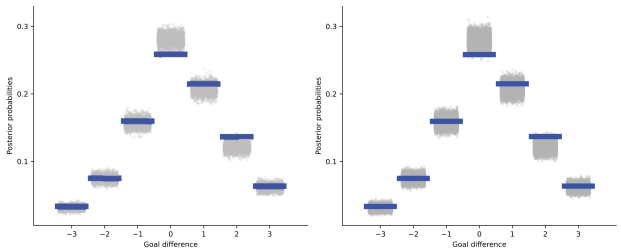


(a) Original article (b) Our replication

Figure 2. Ordered posterior 50% credible intervals for the mixing parameter p_m for each of the 3 420 EPL matches from 2007/08 to 2015/16.

In Figure 2 one can observe the ordered mixing parameter for the 3420 games from the EPL 07/08 to 15/16, where the figure to the left is from the paper, and the figure to the right is from our output. The plots seem to be very much alike. The model favors the betting odds λ_{m_j} (using a lower p_m) when the total number of goals is low. The 277 games with the lowest share of historical data were all games that

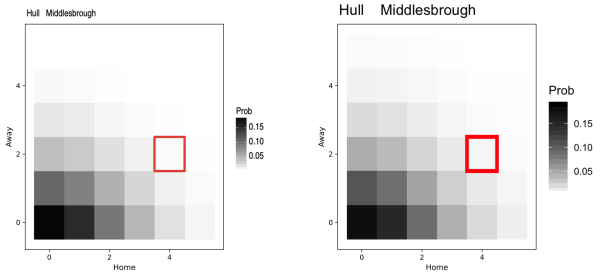
resulted in no goals being scored. The model instead favors the historical match data θ_{m_j} as the number of goals increases. This is possibly due to λ_{m_j} severely underestimating the number of goals scored, caused by the scaling inconsistency addressed earlier. Trying to address this, the historical match data θ_{mj} instead consistently overestimates the number of goals scored. The resulting behavior of the mixing parameter is a negative correlation between the number of goals and the mixing parameter p_m. This is never mentioned in the original paper.



(a) Original article (b) Our replication

Figure 3. Posterior predictive check comparing the observed goal difference $y_1 - y_2$ in the EPL with the replicated goal difference $y_1^{rep} - y_2^{rep}$. The blue bar illustrates the observed share of matches with each goal difference. Gray points represent the share of simulated matches ending with each goal difference.

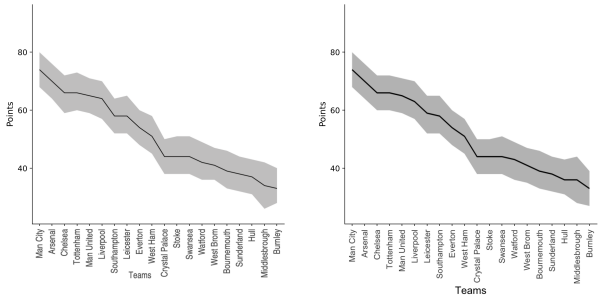
Comparing our replication with the original article in Figure 3, we see that they look very similar. For most of the blue bars, the observed proportion is within the gray cloud, meaning the model manages to reproduce the data well. For both the original paper and our replication, it seems that the model struggles with draws, where a bigger share of draws is produced than what was observed. For home wins by two goals, both the replicated model and the original article also seem to underestimate home wins.



(a) Original article (b) Our replication

Figure 4. Posterior predictive distribution of the final score in Hull–Middlesbrough, EPL 2016/17

Figure 4 shows the posterior predictive distribution of the final score for Hull versus Middlesbrough in the 2016/17 EPL season, where the red square marks the observed result, a 4–2 home win for Hull. Both the original model and our replication predict 0–0 as the most likely outcome, with an associated probability of approximately 0.15. Figure 4 also illustrates the considerable uncertainty involved in predicting individual matches. Although the two distributions are very similar, our replication places somewhat more probability mass on higher-scoring outcomes for both the home and the away team.



(a) Original article (b) Our replication

Figure 5. Predicted final points for each team in the 2016/17 Premier League season. Teams are ordered by predicted points.

Figure 5 shows the posterior predictive of the final amount of points for the test season. The gray bands illustrate the 50% posterior interval for simulating the 380 matches based on the nine previous seasons. The plot from the article and our replication are very similar, with similar medians, similar lengths of the credible bars, and nearly identical ordering of the points. Both the original model and the replicated one show the same teams with the highest and lowest points, and a similar credible interval length of around 15 points.

6. Conclusion

In summary, where the article and the authors JAGS code disagree, we have followed the article, and where the article is unspecific, we follow the code. Some parts of the model could be improved, such as removing the drift in the random walk and placing λ_{mj} on a consistent scale.

Overall, the reproduction in Stan managed to achieve similar results to those reported for the EPL part of the article. The replicated model achieved a model fit without divergences. The posterior probabilities for the top 3 teams gave very similar results along with the figures for the mixing parameter, goal difference, predicted final points, along with the posterior predictive distribution of a final score.

Acknowledgments

We hereby grant our consent for the utilization of this project report as a reference material within the context of future editions of the course.

The posteriordb contribution of this paper are as follows. The processed data for the Premier League (seasons 2007-08 through 2016-17), including the transformed bookmaker odds and match data have been submitted through a pull request. The finished Stan model is also included with this along with the posterior object connecting the two. We have added a reference to the original paper, as well as relevant keywords and descriptions for all objects. A pull request was sent before this paper was submitted. All checks using check_pdb_posterior() pass without any issues.

Appendix

Stan code

```
data {
  // Data concerning matches is entered as a long df, with each row
  // corresponding to a specific match between home and away teams.

  int<lower=1> M;                       // Training matches.
  int<lower=0> M_test;                  // Test matches (season to predict).
  int<lower=2> nteams;                  // Number of teams.
  int<lower=1> T;                       // Number of seasons (train + test).
  int<lower=1> S;                       // Number of bookmakers.

  array[M] int<lower=1, upper=nteams> team1;   // Home team.
  array[M] int<lower=1, upper=nteams> team2;   // Away team.
  array[M] int<lower=1, upper=T> season; // Index each match to a season.
  array[M] int<lower=0> y1;             // Home goals.
  array[M] int<lower=0> y2;             // Away goals.

  // Test matches between home and away teams.
  array[M_test] int<lower=1, upper=nteams> team1_test;
  array[M_test] int<lower=1, upper=nteams> team2_test;

  // Index each test-match to a season.
  array[M_test] int<lower=1, upper=T> season_test;

  // row = match, column = bookmaker.
  // Training data used to fit the model represent seasons 2007/08-2015/16.
  matrix<lower=0>[M, S] theta_bm1;      // Expected goals for home team.
  matrix<lower=0>[M, S] theta_bm2;      // Expected goals for away team.

  // Test data, the data we predict on, season 2016/17.
  matrix<lower=0>[M_test, S] theta_bm1_test; // Expected goals for home team.
  matrix<lower=0>[M_test, S] theta_bm2_test; // Expected goals for away team.

  // Average expected goals over the S bookmakers gives us one value per match.
  // alpha1 for home matches and alpha2 for away matches.
  vector[M] alpha1; // Home team training.
  vector[M] alpha2; // Away team training.
  vector[M_test] alpha1_test; // Home test.
  vector[M_test] alpha2_test; // Away test.
}

parameters {
  real mu;                              // Global intercept of amount of goals.
  real home;                            // Home advantage.
  real<lower=0> sigma2_att;             // Variance of attack effects, positive.
  real<lower=0> sigma2_def;             // Variance of defense effects, positive.
  real mu_att;
  real mu_def;

  // Matrices (teams = rows, columns = season) standardized changes in attack
  // and defense from season to season, used to build a random walk.
  matrix[nteams, T] z_att;
  matrix[nteams, T] z_def;


  // tau_1, tau_2 in eq. 3.7.
  real<lower=0, upper=10> tau1;
  real<lower=0, upper=10> tau2;

  vector<lower=0, upper=1>[M] p;        // Match-specific mixing weights.

  // model estimates it from S bookmakers' values in eq. 3.7.
  // and averages it with historical theta using weight p in eq. 3.3.
  vector<lower=0>[M] lambda1; // Home team, training matches.
  vector<lower=0>[M] lambda2; // Away team, training matches.
  vector<lower=0>[M_test] lambda1_test; // Home team, test matches.
  vector<lower=0>[M_test] lambda2_test; // Away team, test matches.
}

transformed parameters {
  matrix[nteams, T] att;                // Computed attacking and defending.
  matrix[nteams, T] def;                // Strengths in the matrices.
  vector[M] theta1; // Expected home goals based on historical data (team strengths).
  vector[M] theta2; // Expected away goals based on historical data (team strengths).
  vector[M] rate1;  // Expected home goals: mix of historical (theta) and bookmaker (lambda) rates.
  vector[M] rate2;  // Expected away goals: mix of historical (theta) and bookmaker (lambda) rates.
  {
```

```
    // We write the random walk with the drift non-centered.
    matrix[nteams, T] att_star; // Attack strength before zero-sum centring.
    matrix[nteams, T] def_star; // Defense strength before zero-sum centring.

    att_star[, 1] =  mu_att + sqrt(sigma2_att) * z_att[, 1];
    def_star[, 1] =  mu_def + sqrt(sigma2_def) * z_def[, 1];


    for (h in 2:T) {
      att_star[, h] = mu_att + att_star[, h - 1] + sqrt(sigma2_att) * z_att[, h];
      def_star[, h] = mu_def + def_star[, h - 1] + sqrt(sigma2_def) * z_def[, h];
    }
    for (h in 1:T) {
      att[, h] = att_star[, h] - mean(att_star[, h]);
      def[, h] = def_star[, h] - mean(def_star[, h]);
    }
  }
  // Compute the expected number of goals for home and away team in match m for
  // all matches based on historical data.
  for (m in 1:M) {
    theta1[m] = exp(mu + home + att[team1[m], season[m]] + def[team2[m], season[m]]);
    theta2[m] = exp(mu       + att[team2[m], season[m]] + def[team1[m], season[m]]);
  }

  // Likelihood for the number of goals scored requires rate parameters.
  // The rate is modeled as a mix between historical data (theta) and bookmakers'
  // odds (lambda). The mixing parameter p uses a non-informative prior.
  // See eq. 3.3.
  rate1 = p .* theta1 + (1 - p) .* lambda1; // Home team.
  rate2 = p .* theta2 + (1 - p) .* lambda2; // Away team.
}

model {
// Standard normal prior on the standardized changes: later scaled by sqrt(sigma2)
// in transformed parameters, this gives the random walk for team strengths.
  to_vector(z_att) ~ std_normal();
  to_vector(z_def) ~ std_normal();

  // Priors for mu and home.
  mu     ~ normal(0, sqrt(10));
  home   ~ normal(0, sqrt(10));
  mu_att ~ normal(0, sqrt(10));
  mu_def ~ normal(0, sqrt(10));

  // Priors for sigma2 attack and defense.
  sigma2_att ~ inv_gamma(0.001, 0.001);
  sigma2_def ~ inv_gamma(0.001, 0.001);
  p ~ beta(1, 1);                      // p is drawn from a uniform beta.

  // The priors for bookmakers.
  lambda1 ~ lognormal(alpha1, sqrt(10)); // Home team, training matches.
  lambda2 ~ lognormal(alpha2, sqrt(10)); // Away team, training matches.
  lambda1_test ~ lognormal(alpha1_test, sqrt(10)); // Home team, test matches.
  lambda2_test ~ lognormal(alpha2_test, sqrt(10)); // Away team, test matches.

  // Bookmaker likelihood defined in eq 3.7.
  // For each match, and for S bookmakers, the implied scoring rates are
  // defined as lognormal with location lambda and spread tau.
  for (s in 1:S) { // Loop over the bookmakers.
    theta_bm1[, s] ~ lognormal(lambda1, tau1); // Home team, training.
    theta_bm2[, s] ~ lognormal(lambda2, tau2); // Away team, training.
    if (M_test > 0) {
      theta_bm1_test[, s] ~ lognormal(lambda1_test, tau1); // Home team, test.
      theta_bm2_test[, s] ~ lognormal(lambda2_test, tau2); // Away team, test.
    }
  }

  // Using the rates modeled earlier, we have a mix between historical data
  // and bookmakers' odds. Insert this into a Poisson distribution for number
  // of goals scored. Again, see eq.3.3.
  y1 ~ poisson(rate1);
  y2 ~ poisson(rate2);
}

generated quantities {
  // We are interested in storing some quantities concerning predictions.

  // Predicted historical goal rates for the test matches.
  vector[M_test] theta1_test; // Home.
  vector[M_test] theta2_test; // Away.

```

```
    // Predicted number of goals for the test matches.
    array[M_test] int y1_test; // Home.
    array[M_test] int y2_test; // Away.

    array[M] int y1_rep; // The predicted goals, home.
    array[M] int y2_rep; // The predicted goals, away.
    vector[M] log_lik;   // Log likelihood.

    for (m in 1:M_test) {
      // p is chosen from a uniform distribution.
      real p_new = beta_rng(1, 1);

      // We compute thetas as a function of one team's attack in that match,
      // and the other team's defense. Note the inclusion of home team advantage,
      // and a baseline rate of scoring goals mu.
      real th1 = exp(mu + home + att[team1_test[m], season_test[m]] // Home.
                               + def[team2_test[m], season_test[m]]);

      // Same as for the home team, but without advantage.
      real th2 = exp(mu + att[team2_test[m], season_test[m]] // Away.
                        + def[team1_test[m], season_test[m]]);

      // Compute the rates for the number of goals y.
      theta1_test[m] = p_new * th1 + (1 - p_new) * lambda1_test[m];
      theta2_test[m] = p_new * th2 + (1 - p_new) * lambda2_test[m];

      // Insert the final rates and draw a value y.
      y1_test[m] = poisson_rng(theta1_test[m]);
      y2_test[m] = poisson_rng(theta2_test[m]);
    }
    for (m in 1:M) { // Filling the empty output measurements.
      y1_rep[m] = poisson_rng(rate1[m]); // The predicted goals, home.
      y2_rep[m] = poisson_rng(rate2[m]); // The predicted goals, away.

      // Log likelihood.
      log_lik[m] = poisson_lpmf(y1[m] | rate1[m]) + poisson_lpmf(y2[m] | rate2[m]);
    }
  }
```

References

Egidi, L., Pauli, F., and Torelli, N. Combining historical data and bookmakers' odds in modelling football scores. *Statistical Modelling*, 18(5-6):436–459, 2018. doi: 10.1177/1471082X18798414.